

InstANNS: Scalable Approximate Nearest Neighbor Search via Cost-Efficient In-Storage Processing

Bonggeun Sim
Seoul National University
Seoul, South Korea
bg.sim@snu.ac.kr

Yushin Kim
Seoul National University
Seoul, South Korea
henry11n@snu.ac.kr

Minseo Kim
Seoul National University
Seoul, South Korea
ms05251@snu.ac.kr

Yeonhong Park
Seoul National University
Seoul, South Korea
ilil96@snu.ac.kr

Jae W. Lee
Seoul National University
Seoul, South Korea
jaewlee@snu.ac.kr

Abstract

Billion-scale approximate nearest neighbor search (ANNS) increasingly relies on disk-based indexes due to the rapid growth of modern datasets. Existing disk-augmented indexing systems, such as SPANN, often face performance bottlenecks due to limited host interface bandwidth, typically constrained by PCIe. To address this bottleneck, we introduce InstANNS, a storage-centric ANNS architecture that improves throughput and reduces data transfer by performing query-aware PQ filtering inside SSDs, without relying on GPUs. By offloading distance computations to the SSD controller and utilizing abundant internal bandwidth, our design transfers only highly relevant candidates to the host, significantly reducing PCIe traffic. To further optimize performance, we propose *co-occurrence-aware PQ code placement*, which co-locates frequently co-accessed candidates, and *conditional PQ bypass*, which reduces NAND reads by skipping low-utility filtering. We prototype InstANNS by extending SPANN and evaluate it using a device-level SSD simulator, meticulously calibrated against measurements from an actual SSD controller SoC (System-on-Chip) to ensure accurate performance evaluation. Experimental results show that InstANNS improves QPS by 2.15× over SPANN and QPS-per-dollar by 1.7× over FusionANNS at 90% Recall@10, while maintaining accuracy.

CCS Concepts

• **Information systems** → **Retrieval efficiency**; **Storage architectures**; **Query optimization**.

Keywords

Information Retrieval, Approximate Nearest-Neighbor Search, In Storage Processing, Billion Scale Search, High Dimensional Vector

ACM Reference Format:

Bonggeun Sim, Yushin Kim, Minseo Kim, Yeonhong Park, and Jae W. Lee. 2025. InstANNS: Scalable Approximate Nearest Neighbor Search via Cost-Efficient In-Storage Processing. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*, November

10–14, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 11 pages.
<https://doi.org/10.1145/3746252.3761421>

1 Introduction

Approximate Nearest Neighbor Search (ANNS) is a key component in modern retrieval systems, enabling efficient search over billion-scale vector datasets [14, 23, 34, 36]. As data continues to grow, in-memory ANNS methods face high memory costs and limited scalability. To address this, many approaches adopt data compression techniques to reduce memory usage and computation [5, 10, 21]. However, these methods often compromise accuracy and scalability.

Recent systems extend ANNS architectures to storage-based designs that offer cost-effective scalability while maintaining reasonable performance [4, 7, 31, 34, 40]. For example, SPANN [7] adopts an inverted file (IVF)-based approach, where the dataset is partitioned into clusters and each query is matched to a small number of relevant clusters using centroids. SPANN builds an index using these centroids, stores it in memory and store clusters on the solid-state drive (SSD). When a query arrives, it uses the index to identify the candidate clusters. The system then retrieves the corresponding candidate clusters from the SSD for each query, resulting in burst reads. As a result, system throughput scales with available bandwidth, indicating that SSD throughput is the primary performance bottleneck.

To address this bottleneck, various techniques have been proposed to reduce host-device data movement [18, 32]. Among them, FusionANNS [33] represents a state-of-the-art solution. Its core idea, PQ-filtering, reduces the I/O bottleneck of reading vectors by first filtering out irrelevant candidates via approximate distance computation with Product Quantization (PQ), and then retrieving only the filtered vectors from SSD. To implement this idea, FusionANNS stores PQ codes in GPU memory for fast PQ computations, while raw vectors remain on SSD. Although this approach significantly improves QPS and latency, it introduces notable limitations in both cost and resource usage. Specifically, the system relies heavily on GPUs, requiring HBM capacity to hold all PQ codes, which leads to high memory overhead and limits scalability to billion-scale datasets. Moreover, FusionANNS may suffer from read amplification, as fetching the filtered raw vectors from SSD often entails a large number of small, random accesses.

To harness the benefits of PQ filtering while addressing the limitations of FusionANNS, we turn to in-storage processing (ISP)



This work is licensed under a Creative Commons Attribution 4.0 International License.
CIKM '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2040-6/2025/11
<https://doi.org/10.1145/3746252.3761421>

as a means to effectively realize this idea. Modern SSDs typically provide much higher internal bandwidth than the host interface, and PQ filtering is lightweight enough to be executed inside SSDs by leveraging the computational capabilities of the SSD controller, which is achievable through firmware updates without requiring hardware modifications.

In this paper, we propose InstANNS, an ISP-based architecture that leverages PQ filtering for cost-efficient, billion-scale ANNS. Specifically, InstANNS stores both PQ codes and raw vectors inside SSDs and performs PQ filtering entirely within SSDs. While the concept is straightforward, its efficient realization requires careful optimizations, particularly to overcome inefficiencies in accessing PQ codes, which are small and sparsely distributed. To this end, we introduce tailored data layouts and search algorithms that make in-storage PQ filtering practical at scale.

In summary, this paper makes the following contributions:

- We propose a cost-efficient, storage-aware architecture that eliminates the need for GPUs and significantly reduces DRAM usage by offloading computation to SSDs. This architecture fully leverages high internal bandwidth of SSDs through using in-storage processing to improve QPS for billion-scale ANNS.
- We introduce co-occurrence-aware PQ code placement (CoPP), which optimizes data layout to reduce read amplification and minimize unnecessary NAND reads. We also introduce conditional PQ bypass to reduce additional I/Os and improve accuracy. Together, these techniques lead to additional QPS improvements at the same recall.
- We evaluate InstANNS using a device-level SSD simulator based on NVMeVirt, carefully calibrated with a real SSD controller SoC, demonstrating that InstANNS achieves 2.15× higher QPS than SPANN and 1.7× better QPS-per-dollar than FusionANNS at 90% Recall@10, all while preserving accuracy.

2 Background

2.1 Approximate Nearest Neighbor Search

Approximate nearest neighbor search (ANNS) refers to the task of identifying the most similar vector in a large database given a query vector. Broadly, ANNS methods primarily leverage two strategies: (1) retrieving only query-relevant candidates instead of scanning the entire dataset, and (2) compressing data via quantization to reduce memory and computation costs—often combining both to maximize efficiency.

For the first strategy, numerous indexing techniques [11, 12, 16, 17, 27, 28, 37, 41] have been developed to narrow the search space. These techniques reduce the search space by organizing the data into indexing structures or partitioning the high-dimensional space. For example, relevant data can be grouped into clusters, each represented by a centroid, or a graph can be constructed based on data proximity, enabling localized search by traversing neighboring nodes. During query processing, only the subspaces relevant to the given query are explored, significantly improving search efficiency.

The representative approach for the second strategy is product quantization (PQ) [15, 21, 22]. It works by partitioning high-dimensional vectors into multiple low-dimensional subspaces and

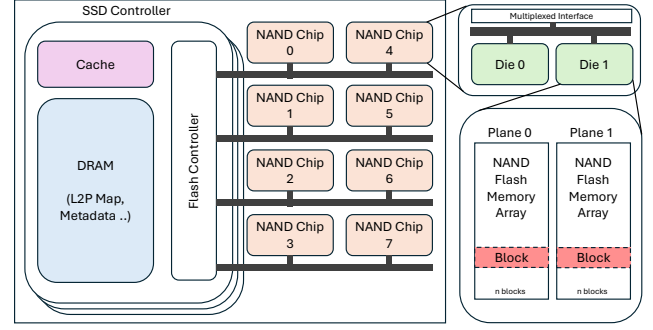


Figure 1: SSD internal architecture

replacing each subvector with the centroid of its nearest cluster, where the clusters are learned in advance via pre-processing. During distance computation with PQ codes, the approximate distance between two vectors is calculated by summing the distances between the corresponding centroids of each subspace. As these centroids remain fixed, the pairwise distances can be precomputed and stored in a lookup table (LUT), enabling PQ to be both memory- and computation-efficient. Given two PQ codes a and b , we compute $PQDist(a, b) = \sum_{i=0}^{M-1} LUT[a_i, b_i]$ in $O(M)$ time, where M is the number of PQ subspaces (the PQ dimension).

2.2 In-Storage Processing with SSDs

In-storage processing (ISP) is gaining traction as a promising approach to offloading computations directly onto SSDs. Unlike traditional architectures, where data must be transferred to the host CPU for processing, ISP enables computation to be performed closer to the data—within the SSD itself. These tasks can be executed by either dedicated hardware or, for lightweight workloads, by the SSD controller alongside its I/O scheduling tasks. The primary benefit of ISP lies in its ability to exploit the overprovisioned internal bandwidth of SSDs, which often exceeds the maximum bandwidth in device specifications.

The internal bandwidth overprovisioning in SSDs stems from their reliance on parallelism to enhance overall bandwidth. As illustrated in Figure 1, SSD architectures incorporate multiple layers of parallelism—across channels, dies, and planes—to address the inherently limited bandwidth of individual NAND flash chips [13, 24]. Coordinating these parallel operations requires embedded controllers, typically equipped with multiple CPU cores to effectively manage concurrent tasks [19, 26]. The theoretical peak internal bandwidth, representing the combined bandwidth of all parallel flash components, can only be fully achieved when accesses are uniformly distributed. However, skewed or irregular access patterns in real-world scenarios rarely allow for such ideal distribution. Therefore, to maintain stable performance under practical workloads [6, 20], SSDs provision internal bandwidth significantly beyond the nominal bandwidth exposed to the host. This extra bandwidth headroom enables SSDs to absorb contention and imbalanced access patterns while meeting host-level performance requirements. Indeed, internal bandwidth commonly exceeds nominal host bandwidth by a factor of 2× to 4× [8], presenting an opportunity to leverage this surplus through carefully designed in-storage processing techniques.

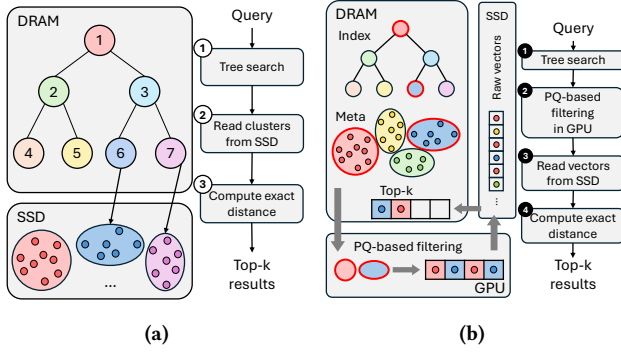


Figure 2: Architecture and query execution process of (a) SPANN (b) FusionANNS

3 Motivation: Toward High-Throughput, Cost-Efficient Large-Scale ANNS

For large-scale ANNS, SSD-based solutions—which store vectors on SSDs instead of in DRAM—are gaining traction as a practical approach. While the largest publicly available ANNS datasets are only a few hundred gigabytes [1, 3, 38], industrial-scale deployments often involve databases spanning several or even tens of terabytes [9, 30]. These sizes typically exceed the capacity of CPU DRAM, necessitating the use of storage devices like SSDs. Although SSDs provide much greater capacity, they suffer from higher latency and lower bandwidth compared to DRAM. To address this gap, SSD-based ANNS solutions leverage system-level optimizations to maximize SSD throughput and minimize latency, alongside algorithmic techniques that reduce I/O overhead [7, 31, 33, 35].

Among various approaches, FusionANNS [33] represents a state-of-the-art SSD-based ANNS solution that significantly reduces I/O traffic by effectively leveraging the PQ technique. Although presenting promising results, FusionANNS reveals notable limitations in implementation that warrant further investigation. Section 3.1 elaborates on the design of FusionANNS along with SPANN, the foundational work it builds upon. Section 3.2 provides a detailed discussion of its limitations.

3.1 SPANN and FusionANNS

SPANN. SPANN [7] is a large-scale IVF-based ANNS system that combines a small in-memory index with large clusters stored on SSD. Figure 2(a) illustrates the architecture and query process of SPANN. During index construction, the dataset is clustered, and a balanced-tree is built using the centroids as the index. This small index is kept in memory, while the full cluster data is stored on SSD. At query time, SPANN performs three steps: ① selecting candidate clusters via the in-memory index, ② retrieving the candidate clusters from the SSD, and ③ computing exact distances to identify the top- k results. This design aims to efficiently utilize SSD bandwidth by performing reads at the granularity of entire clusters, which are typically much larger than the SSD page size (e.g., 4KB).

FusionANNS. Despite SPANN’s SSD-aware design, the limited SSD bandwidth still remains a major bottleneck. To address this, FusionANNS [33] presents an enhanced version of SPANN by introducing

additional techniques to further reduce I/O traffic. Figure 2(b) illustrates the architecture and query process of FusionANNS. The index structure in FusionANNS is identical to SPANN’s, and the query begins similarly: ① candidate clusters are selected via tree search on the in-memory index. The key difference lies in the next stage. Instead of immediately retrieving all raw vectors of candidate clusters from the SSD, FusionANNS applies an additional filtering step. ② It computes PQ distances for all vectors in the candidate clusters and ③ retrieves only a small number of vectors within each candidate cluster with relatively small PQ distances—a step we refer to as *PQ filtering*. ④ It then performs exact distance computations on these subsets to finalize the top- k results.

To implement this process, FusionANNS stores metadata for all clusters (i.e., the vector IDs within each cluster) in DRAM in addition to the index, and leverages the GPU to accelerate the memory-bound PQ distance computations. Precomputed PQ codes for all vectors are stored in GPU HBM, while the SSD holds only the raw vectors.

3.2 Limitations of FusionANNS

FusionANNS significantly improves QPS and latency over SPANN, achieving state-of-the-art performance among SSD-based ANNS systems. However, it suffers from two key limitations: high system cost and SSD read amplification.

Limitation 1: High System Cost. FusionANNS requires relatively large DRAM capacity and heavily relies on GPUs, raising overall system cost. In addition to the index, it stores cluster metadata in DRAM, which can be comparable in size to the index itself. More critically, all PQ codes must reside in the GPU’s HBM, making the GPU the dominant cost driver. As the dataset grows, this design may necessitate multiple GPUs. The original FusionANNS paper reports that the 1.4B SPACEV dataset did not fit on a single GPU with 32 GB of HBM.

Limitation 2 : SSD Read Amplification. While the PQ filtering employed in FusionANNS effectively reduces the amount of data read from SSDs, it introduces a significant drawback: substantial read amplification. Unlike SPANN, which reads data at the granularity of entire clusters, FusionANNS accesses data at the vector granularity, which is much smaller than the standard SSD page size. Although FusionANNS achieves overall performance improvements because the reduction in total I/O outweighs inefficiencies caused by fine-grained reads, this issue remains a critical inefficiency. For example, with the SIFT1B dataset, each raw vector is only 132 bytes, whereas a typical NAND page size is 4 KB, leading to a read amplification factor exceeding 30 $\times$.

4 InstANNS Design

In this section, we present InstANNS, a system designed to overcome the limitations of the existing designs discussed in Section 3, while preserving its key advantage—reduced I/O traffic through PQ filtering. Unlike FusionANNS, InstANNS relies solely on SSDs, requiring no additional DRAM beyond SPANN’s usage or GPU resources. To mitigate potential performance degradation due to the elimination of these additional resources, InstANNS employs in-storage processing. This approach fully exploits the internal bandwidth of SSDs while minimizing data movement between the

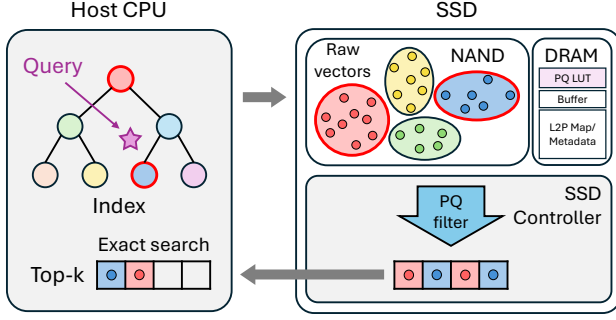


Figure 3: InstANNS overview

CPU and SSD. Furthermore, InstANNS incorporates system-level optimizations that tailor the PQ filtering mechanism to be suitable for in-storage processing.

Section 4.1 provides an overview of InstANNS. Section 4.2 describes the detailed query execution process. Sections 4.3 and 4.4 introduce two optimization techniques that enhance PQ filtering for in-storage processing.

4.1 Overview

Figure 3 illustrates a high-level overview of InstANNS. The system constructs an index following the methodology of SPANN and stores it in CPU DRAM. This index is the only data kept in DRAM, meaning InstANNS introduces no additional memory overhead compared to SPANN. While InstANNS performs PQ filtering similar to FusionANNS, it executes the entire process within the SSD without a GPU. Specifically, both PQ codes and raw vectors are stored in NAND flash inside the SSD, while the PQ LUT is stored in the SSD’s DRAM since it is small and requires fast access. When a query arrives, the SSD controller performs filtering by leveraging the device’s abundant internal bandwidth and returns only the vectors of the filtered candidates to the CPU. As a result, the SSD’s bandwidth is effectively utilized without incurring host-side memory or compute overhead.

Section 4.2 describes the query execution in detail. Sections 4.3 and 4.4 elaborate on two optimization techniques that mitigate the inefficiencies of accessing PQ codes—typically small and sparse—from NAND flash.

4.2 InstANNS Query Execution Process

Figure 4 illustrates the complete execution flow, from query submission at the host to the final result retrieval after in-storage filtering. Upon receiving a query, ❶ the host searches the in-memory index and selects candidate clusters based on a predefined search length. ❷ The host then generates an In-storage Product Quantization Filtering (IPQF) command. Each IPQF command contains the PQ-encoded query, IDs of the candidate clusters (e.g., clusters 0 and 1 in Figure 4), and other necessary metadata. ❸ Upon receiving a command, the SSD controller first issues internal NAND read requests to fetch PQ codes and raw vectors corresponding to the candidate clusters, and ❹ prefetches the PQ LUT from SSD DRAM into the controller cache. These NAND reads and DRAM prefetches are performed in parallel. Once both NAND pages and PQ LUT

entries are available, ❺ the controller performs filtering by computing PQ distances for all vectors within the candidate clusters, maintaining a max-heap to retain the nearest candidates. ❻ The selected vectors, i.e., the retrieval set, are then transferred to the host, completing the command. Finally, ❼ the host performs exact distance computations on the retrieval set and returns the top- k nearest vectors.

4.3 Co-occurrence-Aware PQ Code Placement

We propose the co-occurrence-aware PQ code placement (CoPP) strategy to mitigate excessive random reads and read amplification caused by in-storage filtering. While in-storage filtering reduces host transfer, it introduces a new SSD-side bottleneck: accessing scattered, compact PQ codes results in many small random reads, which waste internal bandwidth and limit performance scalability. The CoPP strategy groups and co-locates PQ codes that are likely to be accessed together, enabling multiple relevant entries to be retrieved with a single, efficient I/O, as illustrated in Figure 5.

Grouping Algorithm. For effective CoPP, it is important to co-locate candidate clusters that tend to be retrieved together by the same query. During query processing, the index determines a set of candidate clusters for each query. Therefore, analyzing which clusters frequently co-occur based on query access patterns enables more effective co-location. Based on this insight, we optimize CoPP by leveraging the index structure of SPANN. The complete algorithm is presented in Algorithm 1.

Step 1: We construct a k -nearest neighbors (kNN) table, which serves as the basis for building the co-occurrence matrix. To construct the kNN table, we simulate queries and use the index to retrieve their nearest neighbors. However, since actual queries are not known in advance, we simulate them by treating each centroid as a pseudo-query. For each pseudo-query, we perform a tree search using the index to retrieve its candidate clusters, which we take as its kNN. The complete set of these results forms the kNN table, from which we compute a co-occurrence matrix that records how frequently each pair of centroids co-appears across the candidate clusters.

Step 2: To construct the co-occurrence matrix efficiently under limited computational and memory resources, we employ a propagation-based co-occurrence analysis strategy. Specifically, starting from a given centroid c , we iteratively traverse its candidate kNNs for up to P propagation steps, collecting a set of centroids that are potentially co-accessed with c .

Step 3: Once the propagation set is identified, we construct the co-occurrence matrix by counting how frequently neighbor pairs co-occur across sampled lists.

Step 4: In our CoPP strategy, we construct groups by greedily clustering centroids with strong co-occurrence. While smaller group size (e.g., 4KB) improve clustering quality, they suffer from lower SSD bandwidth. Conversely, larger group size (e.g., 128KB) are more I/O-efficient but increase read amplification. We parameterize the group size as GS . In our implementation, we set the propagation length $P = 1$ and choose a group size of 16KB, which we empirically found to offer the best tradeoff between locality and I/O efficiency.

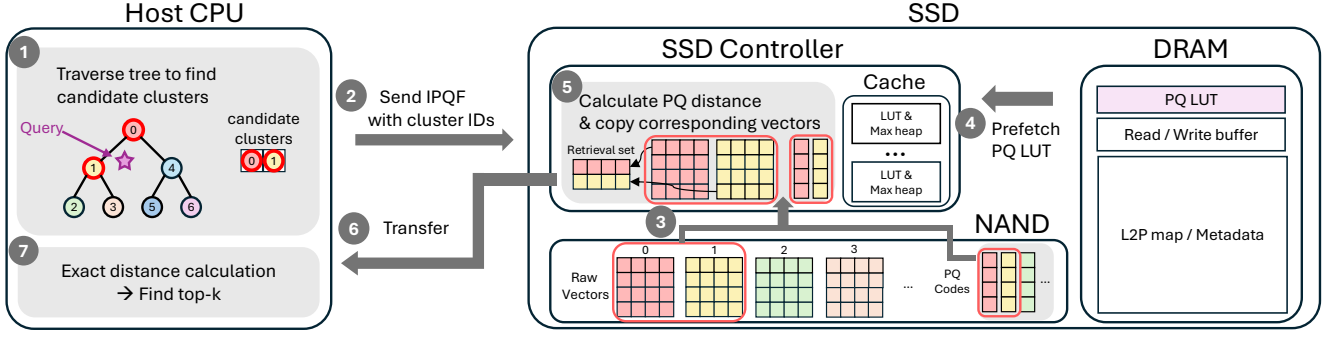


Figure 4: Operations of InstANNS

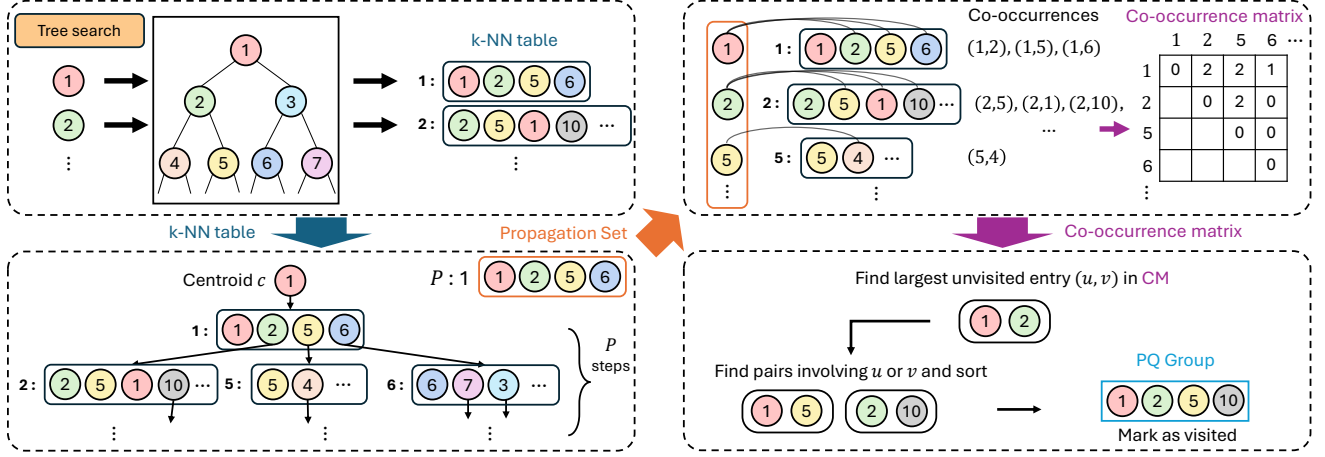


Figure 5: Co-occurrence-Aware PQ Code Placement

4.4 Conditional PQ Bypass

As an additional optimization, we propose a conditional PQ bypass to enhance both recall and QPS. While the CoPP approach effectively reduces NAND reads, it may be suboptimal for certain queries with only a few relevant candidates found in a PQ group. In such cases, accessing the unit triggers two NAND reads—one for the PQ codes and another for the raw vectors—but yields only a limited amount of useful data, resulting in unnecessary NAND read amplification.

We observe that InstANNS opens up an opportunity to improve efficiency when a PQ group contributes only a few candidates. In such cases, bypassing in-storage PQ filtering and directly reading raw vectors—thereby shifting the computation to the host—can be more efficient. This is feasible because InstANNS typically issues small, fixed-size reads (up to 32KB in the SIFT1B dataset), leaving a large portion of host interface bandwidth underutilized. In addition to reducing internal NAND reads, this strategy also improves the utilization of limited heap capacity. By excluding low-contribution PQ groups from InstANNS, the system can allocate more heap entries to groups with higher candidate density, which are more likely to yield relevant results. As a result, more raw vectors are evaluated per query, leading to improved recall.

We formalize this mechanism using a tunable threshold parameter S , which determines whether PQ filtering should be applied

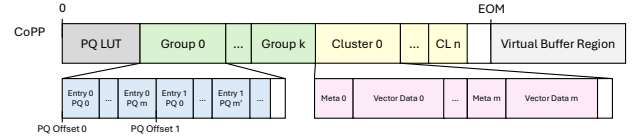


Figure 6: SSD block layout of InstANNS

to a group. If a PQ group contains S or fewer candidate vectors, InstANNS bypasses PQ filtering and directly reads the raw vectors for exact distance computation. While this approach reduces NAND reads, it increases data transfer over the PCIe interface, introducing a trade-off between internal SSD bandwidth and host I/O bandwidth. We evaluate the effect of varying S on this trade-off in Section 6.4 (Figure 9(c)).

5 Implementation

IPQF Specifications. We introduce In-storage Product Quantization Filtering (IPQF), a custom NVMe command that enables query-aware in-storage PQ filtering by encapsulating the query's PQ code, its dimensionality, and per-cluster metadata. We repurpose the NVMe Dataset Management (DSM) command format—originally designed for TRIM operations. Table 1 summarizes the fields: (a)

Algorithm 1 Co-occurrence-Aware PQ Code Placement

```

1: Procedure
2: Input: Tree Index tree
3: Output: PQ Groups PGs

4: Initialize list centroids from tree ▷ Step 1
5: Initialize an empty kNNtable ▷ kNN : k-Nearest Neighbor
6: for each centroid  $c$  in centroids do
7:   kNNlist  $\leftarrow$  tree.search( $c$ )
8:   kNNtable[ $c$ ]  $\leftarrow$  kNNlist

9: Initialize visited as an empty set ▷ Step 2
10: for each entry ( $c$ , kNNlist) in kNNtable do
11:   if  $c$  is in visited then
12:     continue
13:   Initialize set PS with kNNlist ▷ PS : Propagation Set
14:   for  $j = 1$  to  $P$  do ▷ P : Propagation Steps
15:     temp_set  $\leftarrow$  copy of PS
16:     for each  $u$  in temp_set do
17:       for each  $v$  in kNNtable[ $u$ ] do
18:         Add  $v$  to PS

19: Initialize CM as a zero matrix ▷ Step 3
▷ CM : Co-occurrence Matrix
20: for each  $u$  in PS do
21:   for each  $v$  in kNNtable[ $u$ ] do
22:     CM[ $u$ ,  $v$ ]  $\leftarrow$  CM[ $u$ ,  $v$ ] + 1
23: SP  $\leftarrow$  list of ( $u$ ,  $v$ ) pairs by CM[ $u$ ,  $v$ ] descending
▷ SP : Sorted Pairs

24: Initialize PGs as an empty set ▷ Step 4
25: for each ( $u$ ,  $v$ ) in SP do
26:   if  $u$  or  $v$  is in visited then
27:     continue
28:   Create group  $g$  with  $u$  and  $v$ 
29:   Add  $u$  and  $v$  to visited
30:   filtered  $\leftarrow$  all pairs from SP involving  $u$  or  $v$ 
31:   while size( $g$ ) <  $GS$  do
32:      $p \leftarrow$  next pair in filtered
33:      $c \leftarrow$  element in  $p$  that is not  $u$  or  $v$ 
34:     if  $c$  is not in visited then
35:       Add  $c$  to  $g$ 
36:       Add  $c$  to visited
37:   Add  $g$  to PGs

```

SLBA denotes the address of the filtered-result buffer and NLB denotes the size of the filtered vector buffer returned to the host; (b) the command-buffer layout. For each cluster, the metadata include the vector count, the PQ-code offset, and the copy size (including associated metadata such as IDs). DWord 13 specifies the number of valid metadata entries.

SSD Block Layout. Figure 6 shows an example of how to manage database on the SSD. To enable in-storage PQ filtering, the SSD controller must have access to the PQ LUT. Accordingly, we store the LUT in advance in a contiguous SSD address space (e.g., LBA 0–2047) and load it into the controller’s DRAM using a dedicated

Command Dword 10 and 11		Metadata	Bytes	Field
Bits	Description		03:00	Vector Copy Size
63:00	Start LBA (SLBA)		07:04	Reserved
			11:08	PQ Dimensions
			43:12	Query PQ Vectors
Command Dword 12				
Bits	Description			
31:00	Number of Logical Blocks (NLB)	metadata 0	45:44	PQ Offset
			47:46	# of Vectors
			55:48	Start LBA
Command Dword 13				
Bits	Description			
31:16	Reserved		...	
15:00	Number of Ranges	metadata 678	8181:8180	PQ Offset
			8183:8182	# of Vectors
			8191:8184	Start LBA

Table 1: (a) IPQF command specification (b) IPQF buffer specification

initialization command. We store PQ groups and cluster data separately on the SSD, as illustrated in Figure 6. The filtered vector buffer is placed entirely in the controller’s internal memory as a virtual buffer mapped outside the user-visible LBA range, as illustrated in Figure 6. This design prevents unnecessary NAND writes by ensuring that the buffer is never flushed to flash.

Fused Operation. The IPQF command transfers a buffer from the host to the SSD and is treated as a write operation by the SSD. After in-storage filtering, the resulting data must be returned to the host, requiring a subsequent read command. However, the NVMe specification does not permit combining read and write operations within a single command, necessitating that the host must issue these two commands sequentially. In practice, SSDs are designed to accept multiple outstanding commands for performance, which may lead to unintended reordering between the IPQF and the corresponding read command, potentially violating the intended execution order. To ensure correct semantics, these commands must be logically fused. We achieve this by utilizing Dwords 10–12, as shown in Table 1(a), to explicitly link the paired operations.

6 EVALUATION

6.1 Experimental Setup

Simulation Methodology. Since our proposed IPQF command is not supported by existing hardware or storage drivers, we implemented it in NVMeVirt [25], a software-defined NVMe emulator. We extended NVMeVirt to support our custom command and configured it to operate in a conventional SSD mode, emulating NAND-level behavior under realistic SSD channel and PCIe timing models. We assume a 2 $\times$ internal-to-host bandwidth ratio, based on prior work [8] that shows internal bandwidth is often 2 $\times$ –4 $\times$ higher than the nominal host interface bandwidth. A sensitivity study to analyze the impact of varying this ratio is presented in Section 6.4 (Figure 9(a)). To emulate a PCIe Gen4 NVMe SSD, we tuned NVMeVirt accordingly. Using fio, we calibrated both sequential and random read throughput to match typical Gen4 performance, achieving approximately 7000MB/s for sequential reads and 1050KIOPS for 4KB random reads.

To improve the timing fidelity of NVMeVirt, we profiled the actual execution times of critical operations—such as PQ distance computations and LUT prefetching—by deploying our logic on the same ARM-based embedded system-on-chip (SoC) used in Samsung’s research SSD prototype. To achieve high-speed access to PQ LUTs, our design caches them in Tightly Coupled Memory (TCM),

which offers deterministic, single-cycle access [2]. Additionally, we utilize DMA engines to offload memory transfers, further reducing controller overhead. We also pipeline PQ distance computations with NAND reads to enhance controller efficiency. The measured latencies were then used to calibrate NVMeVirt’s internal timing parameters, resulting in a more accurate and realistic modeling of device-level behaviors.

To transmit our custom command without modifying the OS kernel or storage drivers, we integrated the Storage Performance Development Kit (SPDK) [39], which allows user-space I/O submission through a polling-based mechanism. This toolkit provides a flexible and accurate environment for evaluating our command under near-realistic SSD execution models. To prevent core-level interference between components, we bind the NVMeVirt emulator, the SPDK polling thread, and the query processing search thread to separate physical CPU threads. This isolation ensures accurate timing emulation and consistent throughput measurements by avoiding performance degradation due to core contention.

All experiments are conducted using our NVMeVirt-based simulator running on a workstation equipped with an AMD EPYC 7543 CPU (2.8 GHz, 32 cores), 692 GB DDR4 DRAM, and a 4 TB Samsung 990 Pro SSD, without using any GPUs. For isolation, concurrent host writes were prevented and SSD background tasks (garbage collection and block managements) were temporarily turned off during evaluations.

Datasets. We evaluate our method on three widely used billion-scale ANN datasets: SIFT1B, DEEP1B, and SPACEV1B. Table 2 summarizes their key characteristics. To compare InstANNS with other baselines, we fix the DRAM usage to approximately 32GB across all datasets: SIFT1B, SPACEV1B, and DEEP1B.

Dataset	Data Type	Dimension	Data Size
SIFT1B	uint8	128	1B
DEEP1B	float32	96	1B
SPACEV1B	int8	100	1.4B

Table 2: Summary of datasets

Comparison Baselines. We compare our proposed InstANNS with four state-of-the-art billion-scale disk-based ANNS systems. Among these, three systems operate entirely using CPU memory and SSDs without accelerators, while the remaining system, FusionANNS [33], leverages both CPU DRAM and GPU HBM to accelerate filtering.

- **SPANN** [7] is an SSD-based ANNS system using an IVF index. It increases disk space usage to improve performance.
- **DiskANN** [31] is an SSD-based system that employs a disk-resident graph index to achieve high throughput, but suffers from high latency.
- **Starling** [35] builds on DiskANN and improves performance by co-locating neighbors within the same SSD block.
- **FusionANNS** [33] extends SPANN by offloading PQ filtering to GPU, thereby reducing disk I/O.

Due to its reliance on GPU acceleration and high-bandwidth memory, FusionANNS is not directly comparable to CPU-based methods and is evaluated separately.

We also considered other SSD-based methods such as IVF-PQ [21], HM-ANN [29], and SmartANNS [32] for evaluation, but excluded them for the following reasons. IVF-PQ was not included due to its limited recall performance, which makes it unsuitable for recall-QPS evaluation. HM-ANN targets heterogeneous memory systems rather than SSD-centric designs, and the lack of a public implementation prevents fair comparison. The core contribution of SmartANNS is scaling SSD bandwidth through multiple (Smart)SSDs combined with graph partitioning techniques, which is orthogonal to our approach and, in principle, could be applied alongside it.

6.2 Overall Performance

Figure 7 presents the QPS results at Recall@10 across varying recall levels for each dataset. Our method consistently achieves 2.1–3.2× higher QPS than SPANN and demonstrates better scalability at high recall levels. In SPANN, the number of SSD pages accessed grows exponentially with increasing recall targets. In contrast, InstANNS reduces disk I/O through CoPP and conditional PQ bypass, enabling it to overcome the inherent QPS scaling limits imposed by the internal-to-PCIe bandwidth ratio and deliver greater scalability.

In terms of latency, the relative performance between SPANN and DiskANN varies with the recall target. DiskANN, a graph-based method, performs multi-hop traversals, inherently increasing latency. As recall increases, SPANN’s latency increases due to the growing disk I/O required to retrieve more candidates. InstANNS, by reducing NAND reads, maintains consistently low latency across all recall levels, achieving up to 11–38% lower latency compared to SPANN.

Both DiskANN and Starling store PQ codes in memory. However, while DiskANN dynamically computes file offsets during search, Starling stores explicit file offsets for each vector. This design choice forces Starling to use a lower PQ dimension to stay within the same memory budget, leading to degraded recall and overall performance.

Finally, we found that Starling’s public implementation does not support index construction on int8 datasets, thus making evaluation on the SPACEV dataset infeasible. Although we attempted forced index construction, the process required approximately three times the index size in peak memory usage. Given SPACEV’s 1.4B vectors, this led to out of memory errors and repeated failures. Therefore, we exclude Starling from evaluation on this dataset.

Effect of Conditional PQ Bypass. Figure 8 shows how conditional PQ bypass affects both system performance and the number of NAND read operations. The results confirm that reducing NAND reads is associated with increased QPS. In addition, conditional PQ bypass not only reduces NAND reads but also improves recall at a given search length, as discussed in Section 4.4. This implies that the system can achieve a given target recall with a shorter search length. This effect becomes more pronounced at high recall levels, where methods without conditional PQ bypass require significantly more candidate reads. As a result, conditional PQ bypass delivers greater QPS improvements in high-recall scenarios by reducing both NAND reads and search overhead.

6.3 Comparison with GPU-based Approach

In this section, we compare our proposed method, InstANNS, with FusionANNS [33], a representative GPU-based ANNS system. Since

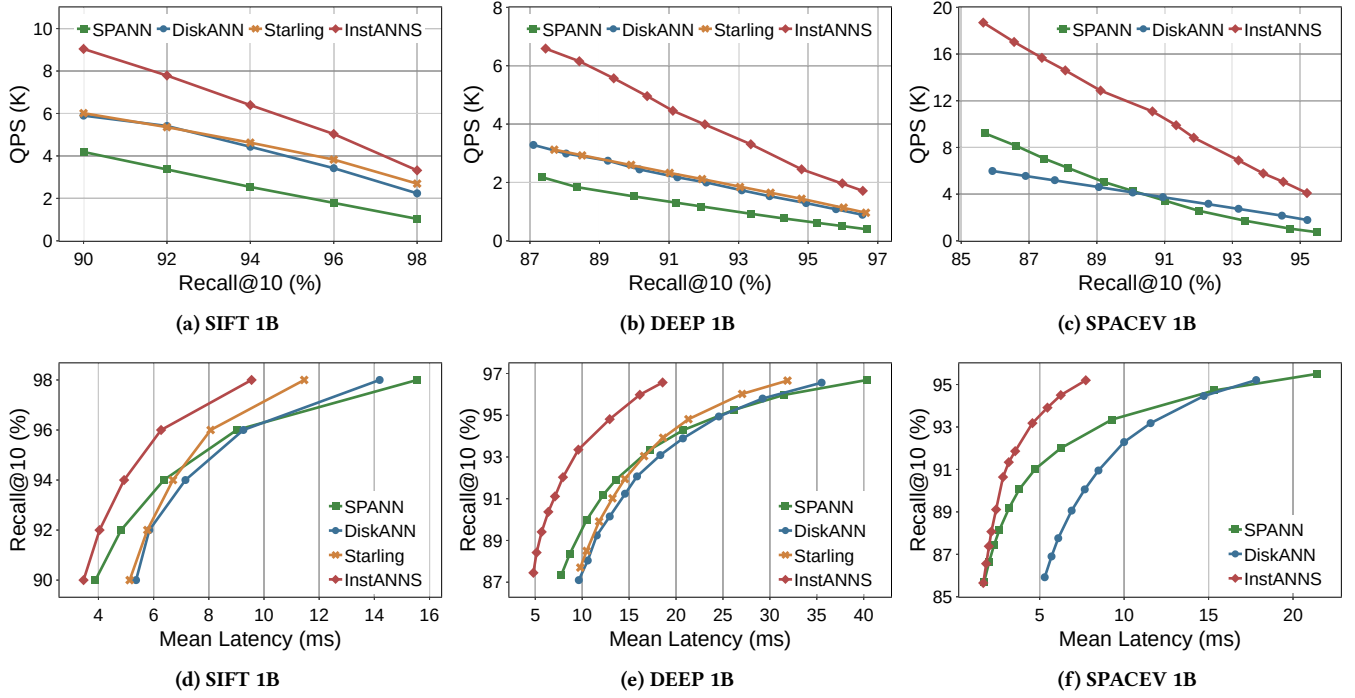


Figure 7: The QPS and latency of different ANNS systems. Higher is better.

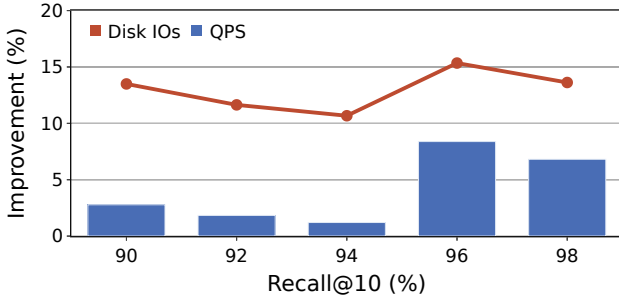


Figure 8: Effect of Conditional PQ Bypass

Recall	QPS (K)			QPS/\$		
	SPANN	Fusion	InstANNS	SPANN	Fusion	InstANNS
90%	4.19	14.3	9.04	2.41	3.07	5.13
92%	3.37	10.7	7.79	1.94	2.29	4.43
94%	2.54	8.55	6.39	1.46	1.83	3.63
96%	1.79	6.23	5.03	1.03	1.33	2.86
98%	1.03	3.72	3.32	0.59	0.79	1.89

Table 4: Cost efficiency for systems

	Name	Price	Note
CPU	AMD EPYC 7543	\$1500	
MEM	DDR5 RDRAM	\$5/GB	32GB for all systems
SSD	990 Pro 4TB	\$0.1/GB	800G for SPANN 100G for FusionANNS 1T for InstANNS
GPU	NVIDIA V100 32G	\$3000	only FusionANNS
Cost	\$1740 for SPANN, \$4670 for FusionANNS, \$1760 for InstANNS		

Table 3: System configurations and costs

the source code for FusionANNS is not publicly available, we re-implemented the system based on the descriptions provided in the original paper. To ensure a fair and conservative comparison,

we applied several optimistic approximations to FusionANNS, enabling it to reach its theoretical maximum performance. Specifically, to approximate FusionANNS’s best-case performance without access to its GPU implementation, we precomputed PQ distance results and loaded them into memory, sorted by distance. This approach eliminates the runtime overhead of PQ computations and yields zero GPU latency. Additionally, to precisely control I/O overhead, we restricted the number of disk reads to match the recall-dependent I/O counts reported in the original paper. Consequently, our re-implementation is expected to significantly outperform the originally reported numbers and serves as an upper bound on FusionANNS’s achievable performance.

Experimental Setup. All systems were evaluated on the SIFT1B dataset with the same number of PQ dimensions, using Recall@10 as the evaluation metric. To ensure fairness, we matched the amount of DRAM allocated to each system, while retaining the original GPU configuration (including HBM) for FusionANNS.

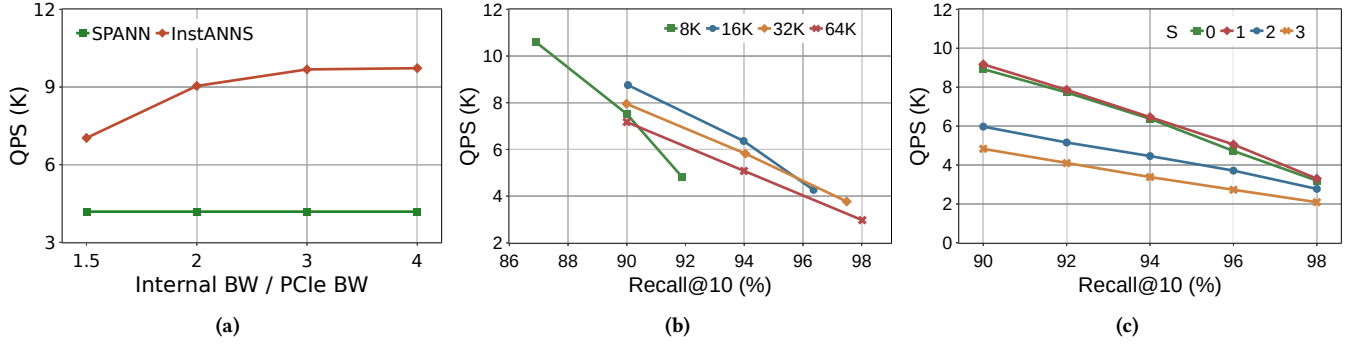


Figure 9: Evaluating the effect of (a) internal bandwidth scaling (b) retrieval set size (c) conditional PQ bypass threshold

Cost Efficiency Comparison. To enable a fair comparison, we evaluated the cost efficiency of both systems using our actual hardware configuration as the baseline. Memory allocation was kept consistent across both systems, and storage cost was calculated based on the actual data utilized by each. For FusionANNS, we adopted the same CPU, DRAM, and storage pricing model, but added GPU cost according to the specifications reported in the original paper. Specifically, we estimate hardware costs based on publicly listed prices on Amazon.com (as of May 24, 2025), as shown in Table 3.

Despite using an idealized upper bound for FusionANNS performance, InstANNS achieves 1.7-2.4 $\times$ higher QPS/\$ across all recall levels, as shown in Table 4. In addition to its superior cost efficiency, InstANNS does not rely on GPU resources, making it more suitable for co-deployment with GPU-intensive applications such as RAG models and recommendation systems.

6.4 Sensitivity Study

Figure 9 presents the effect of varying ratio of SSD internal bandwidth to PCIe bandwidth, retrieval set size, and conditional PQ bypass threshold on end-to-end performance using the SIFT1B dataset.

Effect of Varying Internal Bandwidth. In Figure 9(a), we fix the PCIe bandwidth and vary the SSD internal bandwidth from 1.5-4 $\times$ of the PCIe bandwidth. The results show performance saturation around 3-4 $\times$, suggesting that further increases in internal bandwidth provide diminishing returns. This behavior indicates a shift in the bottleneck from NAND reads to the SSD controller’s computational capacity, likely due to limited pipelining within the controller.

Effect of Retrieval Set Size. Figure 9(b) evaluates how the number of PQ-filtered vectors affects performance by varying the retrieval set size while keeping the search length fixed. In the SIFT1B dataset, the number of vectors retrieved per query increases from 62 to 496 as the retrieval set size grows from 8KB to 64KB. A larger retrieval set size results in more vectors being read from the SSD, increasing the number of NAND read operations and thereby reducing QPS. We also observe that the number of vectors required to maintain target recall varies with the recall level. For instance, at 90% Recall@10, a 16KB retrieval set is sufficient to include all relevant vectors within the fixed search range, making it the most efficient

choice. However, as the target recall increases, the inaccuracy of PQ filtering makes it difficult to retrieve all relevant vectors using only 16KB. This necessitates a larger retrieval set size, which leads to increased disk I/O and introduces a trade-off between QPS and recall.

Effect of Conditional PQ Bypass Threshold. Figure 9(c) shows the effect of varying the conditional PQ bypass threshold S on performance at the same recall level. When $S = 0$, a longer search length is required to maintain recall, which increases NAND reads and slightly reduces QPS. In contrast, as S increases beyond 1, excessive raw vector reads occur, leading to higher PCIe bandwidth usage and reintroducing the PCIe bandwidth bottleneck.

7 Conclusion

In this paper, we present InstANNS, a Cost-Efficient, SSD-centric architecture for large-scale ANNS that performs in-storage PQ filtering. Offloading filtering to the SSD exploits internal bandwidth and reduces PCIe bandwidth consumption. Across our evaluations, InstANNS improves QPS by 2.15 $\times$ over SPANN and QPS-per-dollar by 1.7 $\times$ over FusionANNS at 90% Recall@10, while maintaining accuracy. These results highlight the potential of in-storage processing as a foundation for efficient and scalable ANNS systems.

Looking ahead, query-aware multi-version packing of PQ codes—enabled by inexpensive capacity—can duplicate and co-locate frequently co-selected clusters to further reduce NAND reads.

Acknowledgements

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea Government (MSIT) (RS-2024-00340008), and by the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No. RS-2024-00459774, RISC-V based system software development for open ecosystem of SDR). Jae W. Lee is the corresponding author.

GenAI Usage Disclosure

Some words and expressions suggestions, grammar corrections, LaTeX package/command recommendations, and test case ideas (including boundary conditions) were generated or refined with the assistance of a generative AI tool (e.g., ChatGPT), under the authors’ supervision and final review.

References

- [1] 2010. Datasets for approximate nearest neighbor search. <http://corpus-texmex.irisa.fr/>.
- [2] 2016. ARM developer documentation, Tightly Coupled Memory (TCM). <https://developer.arm.com/documentation/107565/0101/Memory-system/Tightly-Coupled-Memory--TCM-> Accessed: 2025-05.
- [3] 2021. SpaceV1b: A billion-scale vector dataset for text descriptors. <https://github.com/microsoft/SPTAG/tree/master/datasets/SPACEV1B>
- [4] Akhil Arora, Sakshi Sinha, Piyush Kumar, and Arnab Bhattacharya. 2018. HD-index: pushing the scalability-accuracy boundary for approximate kNN search in high-dimensional spaces. *Proc. VLDB Endow.* 11, 8 (April 2018), 906–919. doi:10.14778/3204028.3204034
- [5] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. 2018. Revisiting the Inverted Indices for Billion-Scale Approximate Nearest Neighbors. In *Computer Vision – ECCV 2018: 15th European Conference, Munich, Germany, September 8–14, 2018, Proceedings, Part XII* (Munich, Germany). Springer-Verlag, Berlin, Heidelberg, 209–224. doi:10.1007/978-3-030-01258-8_13
- [6] Feng Chen, Rubao Lee, and Xiaodong Zhang. 2011. Essential roles of exploiting internal parallelism of flash memory based solid state drives in high-speed data processing. In *2011 IEEE 17th International Symposium on High Performance Computer Architecture*. 266–277. doi:10.1109/HPCA.2011.5749735
- [7] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: highly-efficient billion-scale approximate nearest neighbor search. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Red Hook, NY, USA, Article 398, 14 pages.
- [8] Sangyeun Cho, Chanik Park, Hyunok Oh, Sungchan Kim, Yungmin Yi, and Gregory R. Ganger. 2013. Active disk meets flash: a case for intelligent SSDs. In *Proceedings of the 27th International ACM Conference on International Conference on Supercomputing* (Eugene, Oregon, USA) (ICS '13). Association for Computing Machinery, New York, NY, USA, 91–102. doi:10.1145/2464996.2465003
- [9] Common Crawl. 2025. Common Crawl: Open Repository of Web Crawl Data. <https://commoncrawl.org/>. Accessed: 2025-05-22.
- [10] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). arXiv:2401.08281 [cs.LG]
- [11] Cong Fu and Deng Cai. 2016. EFANNA : An Extremely Fast Approximate Nearest Neighbor Search Algorithm Based on kNN Graph. arXiv:1609.07228 [cs.CV] <https://arxiv.org/abs/1609.07228>
- [12] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* 12, 5 (Jan. 2019), 461–474. doi:10.14778/3303753.3303754
- [13] Congming Gao, Liang Shi, Mengying Zhao, Chun Jason Xue, Kaijie Wu, and Edwin H.-M. Sha. 2014. Exploiting parallelism in I/O scheduling for access conflict minimization in flash-based solid state drives. In *2014 30th Symposium on Mass Storage Systems and Technologies (MSST)*. 1–11. doi:10.1109/MSST.2014.6855544
- [14] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2, Article 137 (June 2023), 27 pages. doi:10.1145/3589282
- [15] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 4 (2014), 744–755. doi:10.1109/TPAMI.2013.240
- [16] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB '99)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 518–529.
- [17] Ben Harwood and Tom Drummond. 2016. FANNG: Fast Approximate Nearest Neighbour Graphs. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 5713–5722. doi:10.1109/CVPR.2016.616
- [18] Taiga Ikeda, Daisuke Miyashita, and Jun Deguchi. 2025. On Storage Neural Network Augmented Approximate Nearest Neighbor Search. arXiv:2501.16375 [cs.LG] <https://arxiv.org/abs/2501.16375>
- [19] Junsu Im, Jeonggyun Kim, Seonggyun Oh, Jinhyung Koo, Juhung Park, Hoon Sung Chwa, Sam H. Noh, and Sungjin Lee. 2025. Solid State Drive Targeted Memory-Efficient Indexing for Universal I/O Patterns and Fragmentation Degrees. In *Proceedings of the Twentieth European Conference on Computer Systems* (Rotterdam, Netherlands) (EuroSys '25). Association for Computing Machinery, New York, NY, USA, 974–990. doi:10.1145/3689031.3717478
- [20] Yuhun Jun, Shinhyun Park, Jeong-Uk Kang, Sang-Hoon Kim, and Euseong Seo. 2024. We Ain't Afraid of No File Fragmentation: Causes and Prevention of Its Performance Impact on Modern Flash SSDs. In *22nd USENIX Conference on File and Storage Technologies (FAST 24)*. USENIX Association, Santa Clara, CA, 193–208. <https://www.usenix.org/conference/fast24/presentation/jun>
- [21] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57
- [22] Yannis Kalantidis and Yannis Avrithis. 2014. Locally Optimized Product Quantization for Approximate Nearest Neighbor Search. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*. 2329–2336. doi:10.1109/CVPR.2014.298
- [23] Satyanand Kale. 2024. From Text to Recommendations: How Vector Databases are Revolutionizing Personalized Content Delivery. *International Journal for Research in Applied Science and Engineering Technology* 12 (04 2024), 3376–3387. doi:10.22214/ijraset.2024.59852
- [24] Jeong-Uk Kang, Jin-Soo Kim, Chanik Park, Hyoungjun Park, and Joonwon Lee. 2007. A multi-channel architecture for high-performance NAND flash-based storage system. *Journal of Systems Architecture* 53, 9 (2007), 644–658. doi:10.1016/j.sysarc.2007.01.010
- [25] Sang-Hoon Kim, Jaehoon Shim, Euidong Lee, Seongyeop Jeong, Ilkueon Kang, and Jin-Soo Kim. 2023. NVMeVirt: A Versatile Software-defined Virtual NVMe Device. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*. USENIX Association, Santa Clara, CA, 379–394. <https://www.usenix.org/conference/fast23/presentation/kim-sang-hoon>
- [26] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyang Feng, Simon Garcia de Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-mei Hwu. 2019. DeepStore: In-Storage Acceleration for Intelligent Queries. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) (MICRO '52). Association for Computing Machinery, New York, NY, USA, 224–238. doi:10.1145/3352460.3358320
- [27] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68. doi:10.1016/j.is.2013.10.006
- [28] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (April 2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [29] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 10672–10684. https://proceedings.neurips.cc/paper_files/paper/2020/file/788d986905533aba051261497ecffcb-Paper.pdf
- [30] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. 2022. LAION-5B: an open large-scale dataset for training next generation image-text models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 1833, 17 pages.
- [31] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: fast accurate billion-point nearest neighbor search on a single node. Curran Associates Inc., Red Hook, NY, USA.
- [32] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 1135–1150. <https://www.usenix.org/conference/atc24/presentation/tian>
- [33] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xuechang Zhang, Junhua Zhu, and Yu Zhang. 2025. Towards high-throughput and low-latency billion-scale vector search via CPU/GPU collaborative filtering and re-ranking. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies* (Santa Clara, CA, USA) (FAST '25). USENIX Association, USA, Article 11, 15 pages.
- [34] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xianguyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) (SIGMOD '21). Association for Computing Machinery, New York, NY, USA, 2614–2627. doi:10.1145/3448016.3457550
- [35] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xianguyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proc. ACM Manag. Data* 2, 1, Article 14 (March 2024), 27 pages. doi:10.1145/3639269
- [36] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. doi:10.14778/3476249.3476255
- [37] Ian H. Witten, Alistair Moffat, and Timothy C. Bell. 1999. *Managing gigabytes (2nd ed.): compressing and indexing documents and images*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [38] Artem Babenko Yandex and Victor Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2055–2063. doi:10.1109/CVPR.2016.226

- [39] Ziye Yang, James R. Harris, Benjamin Walker, Daniel Verkamp, Changpeng Liu, Cunyin Chang, Gang Cao, Jonathan Stern, Vishal Verma, and Luse E. Paul. 2017. SPDK: A Development Kit to Build High Performance Storage Applications. In *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. 154–161. doi:10.1109/CloudCom.2017.14
- [40] Minjia Zhang and Yuxiong He. 2019. GRIP: Multi-Store Capacity-Optimized High-Performance Nearest Neighbor Search for Vector Search Engine. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (Beijing, China) (CIKM '19)*. Association for Computing Machinery, New York, NY, USA, 1673–1682. doi:10.1145/3357384.3357938
- [41] Justin Zobel, Alistair Moffat, and Kotagiri Ramamohanarao. 1998. Inverted files versus signature files for text indexing. *ACM Trans. Database Syst.* 23, 4 (Dec. 1998), 453–490. doi:10.1145/296854.277632